

Project1_Monopoly

Philopatair Mallik

CSC 17A _ C++

Spring 2026

42419

Introduction:

For project 1, I made a text-based monopoly game. The game will not show any graphics to the user, only text indicating where the player is and all his options for each turn. This version of the game is best played with the game map on hand to be able to visualize locations and keep track of everybody's location and acquired properties.

About the game:

Monopoly is one if not he most popular board game around the world. It was made as a mockery of the capitalist system but families and people of all ages found out that is enjoyable to play and to rage at all the frustration that comes with the fun.

For Rules and a detailed account for the game here is a link to the game wiki:

https://monopoly.fandom.com/wiki/Main_Page

Versions:

Version0:

- * created the main structure of the program with the main game loop and function prototypes.
- * The main game loop iterates through each player's turn, allowing them to take actions and move around the board until the game is over.
- * The initialization function is a placeholder for setting up the game components (cities, board, chance cards, community chest cards) from external files.
- * The getOrderOfPlay function is the first turn of the game where players roll dice to determine the order of play, and the rolls and player order are sorted in descending order based on the rolls.
- * The handleMenuChoice function is a placeholder for handling the various actions a player can take during their turn based on their menu choice. (will be done in version 1)
- * The movePlayer function simulates rolling the dice and moving the player around the board, including handling double rolls and passing Go. (will be completed in version 1)
- * The locationAction function is a placeholder for handling the action when a player lands on a space on the board (e.g., buying property, paying rent, drawing cards, etc.) based on the type of space. (will be completed in version 1)

Most work was done on the game loop especially the order of rolling the dice and moving the player around the board.

The double roll logic was tricky to implement, especially that at each action done player can go to jail or bankrupt which would end their turn even if they rolled a double.

I had to make sure to check for those conditions after each action and end the turn if they occur, even if the player rolled a double.

I also had to make sure to reset the double roll count when the player does not roll a double or when their turn ends for any reason.

I still need to figure out a way to make a graphical representation of the board and the players' positions on it.

which possibly will be done in later version and will be a representation in the console using ASCII characters to create a visual representation.

probably won't be forced to show every turn but whenever a player chooses to view the board or at the end of each round.

Version 1:

- * made the menu function for the player's turn which will allow them to choose their action during their turn (rolling dice, buying properties, upgrading properties, etc.)

- * made the handleMenuChoice function which will execute the action based on the player's menu choice and will allow them to take multiple actions during their turn until they choose to end their turn or they go bankrupt or go to jail

- * made the movePlayer function which simulates rolling the dice and moving the player around the board, including handling double rolls and passing Go. It also calls the locationAction function to handle the action for landing on the new position.

- * made the locationAction function which will direct the flow to different functions based on the type of space the player lands on (e.g., buy property, pay rent, draw card, etc.)

- * The main game loop was updated to incorporate the menu and player movement logic, allowing players to take actions and move around the board during their turns.

- * made the different functions for handling the various actions a player can take during their turn (e.g., buying properties, upgrading properties, mortgaging properties, etc.) and the various actions that can occur when a player lands on a space (e.g., paying rent, drawing cards, etc.)

- * Pay rent function will handle rent payment when a player lands on an owned property, including calculating the rent based on the property's attributes and the number of houses/hotels, and transferring the money from the player to the owner.

it will also handle tax payment when a player lands on a tax space, including calculating the tax amount and transferring the money from the player to the bank.

* Buy property function will handle the process of buying a property when a player lands on an unowned property, including checking if the player has enough money, transferring the money from the player to the bank, and updating the property's ownership.

* Draw Chance card function will handle drawing a Chance card and executing its effect, including moving the player to a different space, giving or taking money, etc. (will be implemented in version 2)

* Draw Community Chest card function will handle drawing a Community Chest card and executing its effect, including moving the player to a different space, giving or taking money, etc. (will be implemented in version 2)

* Money transaction functions will handle money transactions between players (e.g., rent payment, property purchase, etc.) and between players and the bank (e.g., salary collection, property purchase, etc.), including updating the players' balances and handling low balance situations (e.g., selling property, mortgaging property, declaring bankruptcy, etc.)

* Upgrade property function will handle upgrading properties with houses and hotels, including checking for the required conditions (owning all properties in a color group, having enough money, etc.) and updating the property's attributes and the player's balance accordingly.

* Mortgage property function will handle mortgaging properties, including calculating the mortgage value, updating the property's status, and transferring money from the player to the bank.

* Unmortgage property function will handle unmortgaging properties, including calculating the unmortgage cost (including interest), updating the property's status, and transferring money from the player to the bank.

* Sell property function will handle selling properties to other players or back to the bank, including updating the property's ownership and the player's balance accordingly. (will be implemented in version 2)

* Trade property function will handle trading properties between players, including allowing players to negotiate and agree on the terms of the trade, and updating the properties' ownership and the players' balances accordingly. (will be implemented in version 2)

* Declare bankruptcy function will handle bankruptcy declaration, including transferring the bankrupt player's assets to their creditors (other players or the bank) and removing the player from the game.

* Jail turn function will handle a player's turn when they are in jail, including allowing them to attempt to roll doubles to get out of jail, pay a fine to get out of jail, or use a Get Out of Jail Free card if they have one. (will be implemented in version 2)

* Go to Jail function will handle sending a player to jail when they land on the Go To Jail space or when they roll doubles three times in a row, including updating their position and jail status.

also can be called from the locationAction function when a player lands on the Go To Jail space or from the movePlayer function when a player rolls doubles three times in a row.

or from chance/community chest card effects that send the player to jail.

Version 2:

- Added the initialization function to read city and board data from external files
- added the jailturn function to handle a players's turn when they are in jail
- Added the tradeProperty function to handle trading properties between players
- Added two functions to display a player's properties and to display a one-liner description of a property
- Fixed various bugs and edge cases related to property ownership, rent calculation, and bankruptcy declaration
- removed the sellProperty function for now to simplify the implementation, will be added back in version 3

Version 3:

- Added the initialization function to read city and board data from external files
- added the jailturn function to handle a players's turn when they are in jail
- Added the tradeProperty function to handle trading properties between players
- Added two functions to display a player's properties and to display a one-liner description of a property
- Fixed various bugs and edge cases related to property ownership, rent calculation, and bankruptcy declaration
- removed the sellProperty function for now to simplify the implementation, will be added back in version 3

Version 4:

changed names of some variables and functions for clarity and consistency

	functiongroup	type	function name	sent data	return data
1	Initialization	void	initialization	city*, Board*, int* chanceCards, int* communityChestCards	
2		int	initializePlayers	Player*, int noOfPlayers	no of players
3					
4	deallocation	void	deallocateMemory	Player*, city*, Board*, int* chanceCards, int* communityChestCards	
5		void	deallocateMemory	int* array	
6	main game	void	getOrderOfPlay	Player*, int noOfPlayers	
7		void	mainGameLoop	Player*, city*, Board*, int* chanceCards, int* communityChestCards, i*nt housesAvailable, int* hotelsAvailable, int noOfPlayers	
8		int	showMenu	int menuTemplateIndex	choice
9		void	handleMenuChoice	int choice,int menuTemplateIndex, int playerIndex, Player*, city*, Board*, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int noOfPlayers	
10		void	movePlayer	bool* doubleRoll, int playerIndex, Player*, city*, Board*, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int noOfPlayers	
11		void	locationAction	int playerIndex, int rollTotal, Player*, Board*, City*, int* chanceCards, int* communityChestCards	
12	dice roll	void	rollDice	int* diceRoll	

13			
14		void sortArray	int* array, int size, int value
15	array tools	void sortParallelArray	int *arr, Player*, int size
16		int binarySearch	int* array, int value
17		void randomizeArray	int* array, int size
18		void payRentOrTax	
19		void buyProperty	
20	board Actions	void drawChanceCard	
21		void drawCommunityChestCard	
22		void goToJail	
21	jail process	void jailTurn	
22		void moneyTransaction	
23	money logic	void moneyTransaction	
24		void handleLowBalance	
25		void upgradeProperty	
26		void mortgageProperty	
27	menu Actions	void unMortgageProperty	
28		void sellProperty	
29		void tradeProperty	
30		int declareBankruptcy	total assets value

id	name	propertyID	Type	pos0	pos1	pos2	pos3
0	Go	-1	0	-1	-1	-1	-1
1	Mediterranean Avenue	0	1	-1	-1	-1	-1
2	Community Chest	-1	5	-1	-1	-1	-1
3	Baltic Avenue	1	1	-1	-1	-1	-1
4	Income Tax	-1	6	-1	-1	-1	-1
5	Reading Railroad	22	2	-1	-1	-1	-1
6	Oriental Avenue	2	1	-1	-1	-1	-1
7	Chance	-1	4	-1	-1	-1	-1
8	Vermont Avenue	3	1	-1	-1	-1	-1
9	Connecticut Avenue	4	1	-1	-1	-1	-1
10	Jail	-1	8	-1	-1	-1	-1
11	St. Charles Place	5	1	-1	-1	-1	-1
12	Electric Company	26	3	-1	-1	-1	-1
13	States Avenue	6	1	-1	-1	-1	-1
14	Virginia Avenue	7	1	-1	-1	-1	-1
15	Pennsylvania Railroad	23	2	-1	-1	-1	-1
16	St. James Place	8	1	-1	-1	-1	-1
17	Community Chest	-1	5	-1	-1	-1	-1
18	Tennessee Avenue	9	1	-1	-1	-1	-1
19	New York Avenue	10	1	-1	-1	-1	-1
20	Free Parking	-1	9	-1	-1	-1	-1
21	Kentucky Avenue	11	1	-1	-1	-1	-1
22	Chance	-1	4	-1	-1	-1	-1
23	Indiana Avenue	12	1	-1	-1	-1	-1
24	Illinois Avenue	13	1	-1	-1	-1	-1
25	B & O Railroad	24	2	-1	-1	-1	-1
26	Atlantic Avenue	14	1	-1	-1	-1	-1
27	Ventnor Avenue	15	1	-1	-1	-1	-1
28	Water Works	27	3	-1	-1	-1	-1
29	Marvin Gardens	16	1	-1	-1	-1	-1
30	Go To Jail	-1	10	-1	-1	-1	-1
31	Pacific Avenue	17	1	-1	-1	-1	-1
32	North Carolina Avenue	18	1	-1	-1	-1	-1
33	Community Chest	-1	5	-1	-1	-1	-1
34	Pennsylvania Avenue	19	1	-1	-1	-1	-1
35	Short Line	25	2	-1	-1	-1	-1
36	Chance	-1	4	-1	-1	-1	-1
37	Park Place	20	1	-1	-1	-1	-1
38	Luxury Tax	-1	7	-1	-1	-1	-1
39	Boardwalk	21	1	-1	-1	-1	-1

Board		upgrade										
id	Position	City Name	Groupld	Price	cost 1	rent 1	rent 2	rent 3	rent 4	rent 5	Mortgage	Type
0	1	Mediterranean Avenue	0	60	50	2	30	90	160	250	30	1
1	3	Baltic Avenue	0	60	50	4	60	180	320	450	30	1
2	6	Oriental Avenue	1	100	50	6	90	270	400	550	50	1
3	8	Vermont Avenue	1	100	50	6	90	270	400	550	50	1
4	9	Connecticut Avenue	1	120	50	8	100	300	450	600	60	1
5	11	St. Charles Place	2	140	100	10	150	450	625	750	70	1
6	13	States Avenue	2	140	100	10	150	450	625	750	70	1
7	14	Virginia Avenue	2	160	100	12	180	500	700	900	80	1
8	16	St. James Place	3	180	100	14	200	550	750	950	90	1
9	18	Tennessee Avenue	3	180	100	14	200	550	750	950	90	1
10	19	New York Avenue	3	200	100	16	220	600	800	1000	100	1
11	21	Kentucky Avenue	4	220	150	18	250	700	875	1050	110	1
12	23	Indiana Avenue	4	220	150	18	250	700	875	1050	110	1
13	24	Illinois Avenue	4	240	150	20	300	750	925	1100	120	1
14	26	Atlantic Avenue	5	260	150	22	330	800	975	1150	130	1
15	27	Ventnor Avenue	5	260	150	22	330	800	975	1150	130	1
16	29	Marvin Gardens	5	280	150	24	360	850	1025	1200	140	1
17	31	Pacific Avenue	6	300	200	26	390	900	1100	1275	150	1
18	32	North Carolina Avenue	6	300	200	26	390	900	1100	1275	150	1
19	34	Pennsylvania Avenue	6	320	200	28	450	1000	1200	1400	160	1
20	37	Park Place	7	350	200	35	500	1100	1300	1500	175	1
21	39	Boardwalk	7	400	200	50	600	1400	1700	2000	200	1
22	5	Reading Railroad	8	200	NA	25	50	100	200	NA	100	2
23	15	Pennsylvannia Railroad	8	200	NA	25	50	100	200	NA	100	2
24	25	B & O Railroad	8	200	NA	25	50	100	200	NA	100	2
25	35	Short Line	8	200	NA	25	50	100	200	NA	100	2
26	12	Electric Company	9	200	NA	1x	4x	NA	NA	NA	75	3
27	28	Water Works	9	200	NA	1x	4x	NA	NA	NA	75	3

20. Free Darkin	21. Kentucky Avenue - \$220	22. Chance	23. Indiana Avenue - \$220	24. Illinois Avenue - \$240	25. B & O Railroad - \$200	26. Atlantic Avenue - \$260	27. Ventnor Avenue - \$260	28. Water Works - \$150	29. Marvin Gardens - \$280	30. Go To Jail
19. New York Avenue - \$200	MONOPOLY									31. Pacific Avenue - \$300
18. Tennessee Avenue - \$180										32. North Carolina Avenue - \$300
17. Community Chest										33. Community Chest
16. St James Place - \$180										34. Pennsylvani a Avenue - \$320
15. Pennsylvani a Railroad - \$200										35. Short Line - \$200
14. Virginia Avenue										36. Chance
13. States Avenue - \$140										37. Park Place - \$350
12. Electric Company - \$150										38. Luxury Tax
11. St. Charles Place - \$140										39. Broadwalk - \$400
10. Jail	9. Connecticut Avenue - \$120	8. Vermont Avenue - \$100	7. Chance	6. Oriental Avenue - \$100	5. Reading Railroad - \$200	4. Income Tax - Pay \$200	3. Baltic Avenue - \$60	2. Community Chest	1. Mediterranean Avenue - \$60	0. Go

Author: Philopatair Mallik
Date: 05/05/2026
purpose: Project 1 - Monopoly Game Simulation

//system libraries
#include //Input/Output library
#include //Random Number Library
#include //Time Library
#include //Format Library
#include //File I/O Library

//User libraries
#include "Cities.h"
#include "Players.h"
#include "Board.h"

//Global Constants - Physics/Chemistry/Conversions/Array Dimensions
int const MAX_BOARD_SPACES = 40; //Total number of spaces on the board
int const MAX_PLAYERS = 4; //Maximum number of players
int const MAX_CITIES = 22+4+2; //22 properties + 4 railroads + 2 utilities
int const MAX_CHANCE_CARDS = 16; //Number of Chance cards
int const MAX_COMMUNITY_CHEST_CARDS = 16; //Number of Community Chest cards
int const BOARD_SIZE = 40; //Total number of spaces on the board
int const STARTINGBALANCE = 1500; //Starting balance for each players
int const SALARY = 200; //Amount collected when passing Go

//Function Prototype
void initialization(string* filesNames, City* cities, Board* board, int* chanceCards, int* communityChestCards);
City readCityFromFile(fstream &inFile, int id);
Board readBoardFromFile(fstream &inFile, int id);
void deallocateMemory(Player* players, City* cities, Board* board, int* chanceCards, int* communityChestCards);
void deallocateMemory(int* array);
void deallocateMemory(string* array);
int initializePlayers(Player* players, int nOfPlyrs);
void getOrderOfPlay(Player* players, int nOfPlyrs);
void sortParallelArray(int *arr, Player* players, int size);
int binarySearch(int *array, int value);
void sortArray(int *array, int size);
void randomizeArray(int *array, int size);
void rollDice(int* diceRoll)
void mainGameLoop(Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
int showMenu(int menuTemplateIndex);
void handleMenuChoice(int choice, int menuTemplateIndex, int playerIndex, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
void movePlayer(int* doublecount, bool* doubleRoll, int playerIndex, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards, int* housesAvailable, int* hotelsAvailable, int nOfPlyrs);
void locationAction(int playerIndex, int rollTotal, Player* players, Board* board, City* cities, int* chanceCards, int* communityChestCards);
void payRentOrTax(int playerIndex, int rollTotal, Player* player, Board* board, City* cities);
void buyProperty(int playerIndex, Player* players, Board* board, City* cities);
void drawChanceCard(int playerIndex, Player* players, int* chanceCards, Board* board, City* cities);
void drawCommunityChestCard(int playerIndex, Player* players, int* communityChestCards, Board* board, City* cities);
void goToJail(int playerIndex, Player* players);
void jailTurn(int playerIndex, Player* players, Board* board, City* cities);
void moneyTransaction(int fromPlayerIndex, int toPlayerIndex, Player* players, int amount);
void moneyTransaction(int playerIndex, Player* players, int amount);
void handleLowBalance(int playerIndex, int amount, Player* players, Board* board, City* cities);
void upgradeProperty(int playerIndex, int* housesAvailable, int* hotelsAvailable, Player* players, Board* board, City* cities);
void mortgageProperty(int playerIndex, Player* players, Board* board, City* cities);
void unmortgageProperty(int playerIndex, Player* players, Board* board, City* cities);
void sellProperty(int playerIndex, Player* players, Board* board, City* cities);
void tradeProperty(int playerIndex, Player* players, Board* board, City* cities);
int declareBankruptcy(int playerIndex, int* housesAvailable, int* hotelsAvailable, Player* players, Board* board, City* cities);
void calculateGroupOwned(int propertyID, int playerIndex, Player* players, City* cities);
void listPlayerProperties(int playerIndex, Player* players, City* cities);
void displayPropertyOneLiner(int propertyID, City* cities);





